

Mini Store Manager

Rapport de projet : Advanced SQL Development Project (MCS DE1)

Ania Sadoudi

1. Vue d'ensemble du projet

Objectif

L'application « Mini Store Manager » permet à un petit commerce local de gérer son catalogue de produits, sa base clients et ses commandes, et de consulter des indicateurs de vente calculés directement via des requêtes SQL sur une base SQLite.

Technologie choisie

- Backend : Python 3 / Flask
- Base de données : SQLite
- Frontend : templates Jinja2 (rendu côté serveur) + Bootstrap 5 via CDN
- Driver base de données : module sqlite3 natif de Python, requêtes SQL brutes et paramétrées

2. Conception de la base de données

Le schéma comprend les 5 tables imposées par le brief : Category, Product, Customer, CustomerOrder et OrderItem.

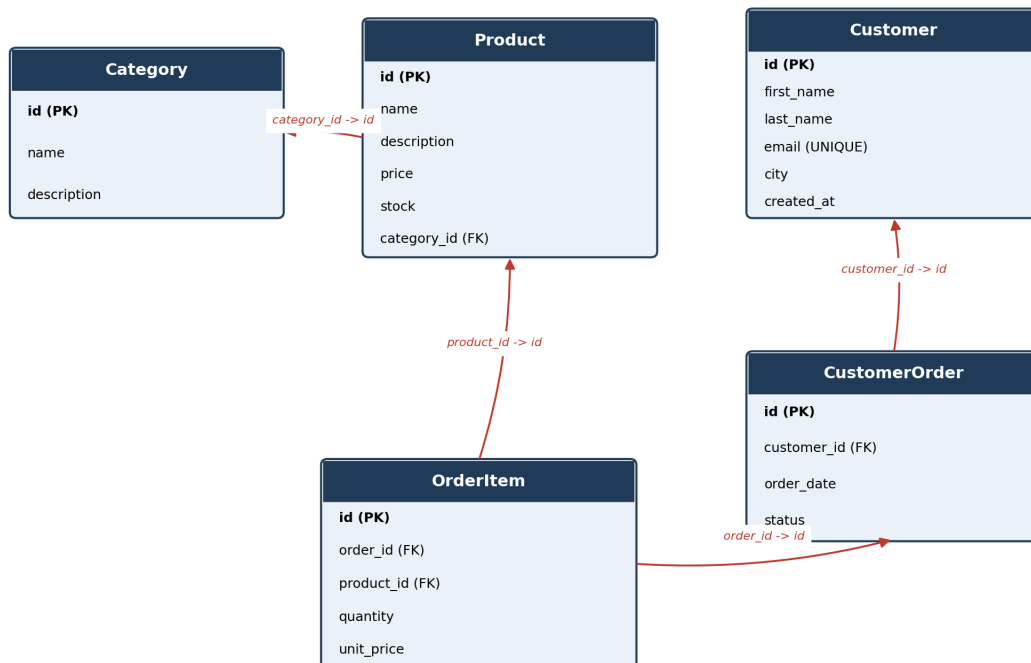
- Category (1) → Product (N) : chaque produit appartient à une catégorie.
- Customer (1) → CustomerOrder (N) : chaque commande est passée par un client.
- CustomerOrder (1) → OrderItem (N) et Product (1) → OrderItem (N) : OrderItem est la table associative qui résout la relation N-N entre commandes et produits (une commande contient plusieurs produits, un produit apparaît dans plusieurs commandes).

Contraintes appliquées

- Clé primaire auto-incrémentée (INTEGER PRIMARY KEY) sur chaque table
- Clés étrangères explicites, activées à chaque connexion via PRAGMA foreign_keys = ON
- NOT NULL sur tous les champs obligatoires
- UNIQUE sur Customer.email
- CHECK (price >= 0) et CHECK (stock >= 0) sur Product
- CHECK (quantity > 0) et CHECK (unit_price >= 0) sur OrderItem

Diagramme

Mini Store Manager - Modele relationnel (ER Diagram)



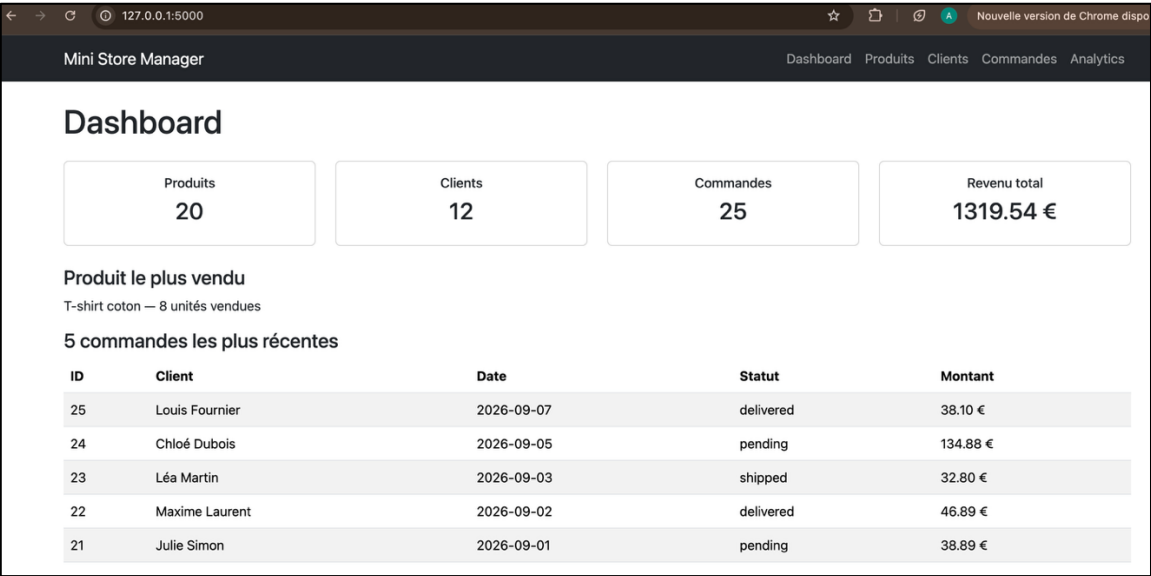
3. Architecture de l'application

```
mini-store-manager/
├── database/
│   ├── schema.sql      (structure : 5 tables, contraintes, clés)
│   ├── seed.sql        (données de test)
│   └── store.db         (généré à partir des scripts, non versionné)
├── src/
│   ├── app.py           (routes Flask, logique métier)
│   ├── db.py            (connexion SQLite)
│   └── templates/       (templates HTML Jinja2)
├── report/
│   └── report.pdf
├── requirements.txt
└── README.md
```

La connexion à la base est ouverte à chaque requête HTTP via la fonction `get_db()` (dans `db.py`), qui active les clés étrangères et configure `row_factory = sqlite3.Row` pour accéder aux colonnes par nom dans les templates. Chaque route Flask ouvre sa propre connexion et la ferme explicitement en fin de traitement. Il n'y a pas de connexion partagée persistante entre requêtes.

4. Pages de l'application

Dashboard



Products

127.0.0.1:5000/products

Mini Store Manager

Dashboard Produits Clients Commandes Analytics

Produits

+ Nouveau produit

Nom	Catégorie	Prix	Stock	Actions
Ballon de football	Loisirs & Jeux	16.90 €	40	Éditer Supprimer
Bougie parfumée	Maison & Jardin	11.50 €	70	Éditer Supprimer
Café en grains 1kg	Alimentation	12.50 €	100	Éditer Supprimer
Casque audio	Informatique	39.99 €	4 Stock bas	Éditer Supprimer
Casquette	Vêtements	12.90 €	45	Éditer Supprimer
Chaussettes (lot de 3)	Vêtements	9.90 €	2 Stock bas	Éditer Supprimer
Chocolat noir 70%	Alimentation	3.50 €	3 Stock bas	Éditer Supprimer
Clavier mécanique	Informatique	59.99 €	30	Éditer Supprimer
Coussin décoratif	Maison & Jardin	18.90 €	35	Éditer Supprimer
Jeu de cartes	Loisirs & Jeux	7.90 €	1 Stock bas	Éditer Supprimer
Jeu de société familial	Loisirs & Jeux	29.90 €	18	Éditer Supprimer
Lampe de bureau	Maison & Jardin	24.90 €	4 Stock bas	Éditer Supprimer

Customers

127.0.0.1:5000/customers

Nouvelle version de Chrome dispon

Mini Store Manager

DashboardProduitsClientsCommandesAnalytics

Clients

+ Nouveau client

Rechercher (nom, email)

Rechercher

Nom	Email	Ville	Nb commandes	Total dépensé
Hugo Bernard	hugo.bernard@email.com	Lyon	3	184.28 €
Chloé Dubois	chloe.dubois@email.com	Marseille	3	237.77 €
Louis Fournier	louis.fournier@email.com	Nantes	3	151.69 €
Camille Girard	camille.girard@email.com	Strasbourg	2	62.30 €
Maxime Laurent	maxime.laurent@email.com	Rennes	2	127.58 €
Thomas Leroy	thomas.leroy@email.com	Lyon	0	0.00 €
Léa Martin	lea.martin@email.com	Paris	4	203.85 €
Sarah Michel	sarah.michel@email.com	Paris	0	0.00 €
Enzo Morel	enzo.morel@email.com	Bordeaux	1	75.79 €
Nathan Petit	nathan.petit@email.com	Toulouse	2	126.10 €
Manon Roux	manon.roux@email.com	Nice	1	46.40 €
Julie Simon	julie.simon@email.com	Lille	2	103.78 €

Orders

127.0.0.1:5000/orders

Nouvelle version de Chrome disponible

Mini Store Manager

DashboardProduitsClientsCommandesAnalytics

Commandes

+ Nouvelle commande

ID	Client	Date	Statut	Montant	
25	Louis Fournier	2026-09-07	delivered	38.10 €	Voir
24	Chloé Dubois	2026-09-05	pending	134.88 €	Voir
23	Léa Martin	2026-09-03	shipped	32.80 €	Voir
22	Maxime Laurent	2026-09-02	delivered	46.89 €	Voir
21	Julie Simon	2026-09-01	pending	38.89 €	Voir
20	Enzo Morel	2026-08-25	cancelled	22.00 €	Voir
19	Camille Girard	2026-08-23	delivered	29.40 €	Voir
18	Hugo Bernard	2026-08-20	shipped	42.70 €	Voir
17	Louis Fournier	2026-08-18	delivered	73.89 €	Voir
16	Manon Roux	2026-08-15	delivered	46.40 €	Voir
15	Nathan Petit	2026-08-12	pending	76.20 €	Voir
14	Léa Martin	2026-08-10	delivered	74.87 €	Voir

Analytics

Mini Store Manager | Dashboard | Produits | Clients | Commandes | Analytics

Analytics

A — Revenu par catégorie

Catégorie	Unités vendues	Revenu
Informatique	16	619.84 €
Vêtements	13	227.70 €
Loisirs & Jeux	11	204.90 €
Maison & Jardin	8	137.10 €
Alimentation	17	130.00 €

B — Top 5 clients

Client	Nb commandes	Total dépensé
Chloé Dubois	3	237.77 €
Léa Martin	4	203.85 €
Hugo Bernard	3	184.28 €
Louis Fournier	3	151.69 €
Maxime Laurent	2	127.58 €

C — Produits jamais commandés

Produit

5. SQL : deux requêtes avancées de la page Analytics

Requête 1 : Produits jamais commandés (NOT EXISTS)

```
SELECT p.id, p.name
FROM Product p
WHERE NOT EXISTS (
    SELECT 1 FROM OrderItem oi WHERE oi.product_id = p.id
);
```

Cette requête sert à trouver les produits qui n'apparaissent dans aucune ligne de OrderItem. Pour chaque ligne de Product, une sous-requête corrélée vérifie s'il existe au moins une ligne dans OrderItem référençant ce produit. NOT EXISTS retourne vrai quand la sous-requête ne trouve aucune ligne correspondante, le produit est alors conservé dans le résultat.

Requête 2 : Commandes au-dessus de la moyenne (CTE)

```
WITH order_totals AS (
    SELECT co.id AS order_id,
           SUM(oi.quantity * oi.unit_price) AS total_amount
    FROM CustomerOrder co
    JOIN OrderItem oi ON co.id = oi.order_id
    GROUP BY co.id
)
SELECT order_id, total_amount
FROM order_totals
WHERE total_amount > (SELECT AVG(total_amount) FROM order_totals);
```

Cette requête a besoin du montant total de chaque commande à deux endroits différents : une fois comme ensemble de lignes à filtrer, une autre fois pour calculer la moyenne qui sert de seuil de comparaison. Le WITH order_totals AS (...) nomme le résultat intermédiaire une seule fois, comme une table virtuelle réutilisable, la requête externe et la sous-requête AVG(...) s'appuient toutes les deux dessus.

6. Conclusion

Difficultés rencontrées

- Construire un schéma relationnel cohérent avec les bonnes contraintes (clés étrangères, CHECK anti-valeurs négatives) dès la conception, pour éviter d'avoir à les ajouter après coup.
- Comprendre l'effet d'un double LEFT JOIN sur le comptage : après une jointure Customer, CustomerOrder, OrderItem, une même commande apparaît sur plusieurs lignes, ce qui fausse un COUNT().

- Garantir l'atomicité de la création de commande pour éviter qu'un échec partiel ne laisse un stock décrémenté sans commande enregistrée, ou l'inverse.

Limites actuelles

- Pas d'authentification.
- Interface minimale, basée sur Bootstrap par défaut, sans design personnalisé.
- Pas de gestion de la concurrence : deux commandes simultanées sur le même produit pourraient, en théorie, dépasser le stock disponible.

Améliorations possibles

- Pagination sur les listes (produits, clients, commandes) si le volume de données augmente.
- Export CSV des analyses de la page Analytics.
- Authentification pour un usage multi-utilisateurs.
- Tests automatisés (actuellement l'application a été testée manuellement, page par page).